

Database Design for Option Pricing Application

Overview

This document outlines the database design for an option pricing application that serves 100 traders making approximately 20 requests per day (2,000 total requests/day). The application compares Pareto (Power Law), Black-Scholes-Merton, and market prices for tail options.

Load Analysis

- **Users:** 100 traders
- **Daily Requests:** 20 requests per trader = 2,000 requests/day
- **Peak Load:** Assuming market hours (8 hours), ~250 requests/hour or ~4 requests/minute
- **Data Volume:** Low to moderate, primarily numerical calculations and pricing data

Database Choice: SQL (PostgreSQL)

Why SQL over NoSQL?

Given the application requirements, **SQL (PostgreSQL)** is the recommended choice for the following reasons:

1. **Structured Financial Data:** Option pricing involves highly structured, relational data with clear relationships between traders, options, prices, and market data
2. **ACID Compliance:** Financial applications require strong consistency and transactional integrity
3. **Complex Queries:** Need for aggregations, joins, and analytical queries on pricing data
4. **Moderate Scale:** 2,000 requests/day is well within SQL database capabilities
5. **Reporting Requirements:** Financial applications typically need complex reporting and analytics
6. **Data Integrity:** Strong schema enforcement prevents data corruption in financial calculations

Database Schema Design

Core Tables

1. Traders Table

```
CREATE TABLE traders (  
  trader_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  full_name VARCHAR(100) NOT NULL,  
  organization VARCHAR(100),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  last_login TIMESTAMP,  
  is_active BOOLEAN DEFAULT TRUE  
);
```

```
CREATE INDEX idx_traders_username ON traders(username);
CREATE INDEX idx_traders_email ON traders(email);
```

2. Market Data Table

```
CREATE TABLE market_data (
  market_data_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  symbol VARCHAR(10) NOT NULL,
  spot_price DECIMAL(10,2) NOT NULL,
  risk_free_rate DECIMAL(6,4) NOT NULL,
  data_date DATE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE(symbol, data_date)
);

CREATE INDEX idx_market_data_symbol_date ON market_data(symbol, data_date);
```

3. Option Contracts Table

```
CREATE TABLE option_contracts (
  contract_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  symbol VARCHAR(10) NOT NULL,
  strike_price DECIMAL(10,2) NOT NULL,
  expiration_date DATE NOT NULL,
  option_type VARCHAR(4) CHECK (option_type IN ('CALL', 'PUT')) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE(symbol, strike_price, expiration_date, option_type)
);

CREATE INDEX idx_option_contracts_symbol ON option_contracts(symbol);
CREATE INDEX idx_option_contracts_expiration ON
option_contracts(expiration_date);
```

4. Pricing Requests Table

```
CREATE TABLE pricing_requests (
  request_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trader_id UUID NOT NULL REFERENCES traders(trader_id),
  contract_id UUID NOT NULL REFERENCES option_contracts(contract_id),
  market_data_id UUID NOT NULL REFERENCES market_data(market_data_id),

  -- Input Parameters
  volatility DECIMAL(6,4) NOT NULL,
  time_to_expiry DECIMAL(8,6) NOT NULL,
```

```

    tail_index_alpha DECIMAL(4,2) NOT NULL,
    anchor_strike DECIMAL(10,2),
    anchor_price DECIMAL(10,2),

    -- Calculated Prices
    bsm_price DECIMAL(10,4),
    pareto_price DECIMAL(10,4),
    market_reference_price DECIMAL(10,4),

    -- Metadata
    request_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    calculation_status VARCHAR(20) DEFAULT 'PENDING' CHECK
(calculation_status IN ('PENDING', 'COMPLETED', 'FAILED')),
    error_message TEXT
);

CREATE INDEX idx_pricing_requests_trader_id ON pricing_requests(trader_id);
CREATE INDEX idx_pricing_requests_timestamp ON
pricing_requests(request_timestamp);
CREATE INDEX idx_pricing_requests_status ON
pricing_requests(calculation_status);

```

5. Price Comparisons Table

```

CREATE TABLE price_comparisons (
    comparison_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id UUID NOT NULL REFERENCES pricing_requests(request_id),

    -- Price Differences
    pareto_vs_bsm_diff DECIMAL(10,4),
    pareto_vs_bsm_pct DECIMAL(6,2),
    market_vs_bsm_diff DECIMAL(10,4),
    market_vs_bsm_pct DECIMAL(6,2),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_price_comparisons_request_id ON
price_comparisons(request_id);

```

6. User Sessions Table

```

CREATE TABLE user_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trader_id UUID NOT NULL REFERENCES traders(trader_id),
    session_start TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    session_end TIMESTAMP,
    ip_address INET,

```

```
    user_agent TEXT,  
    is_active BOOLEAN DEFAULT TRUE  
);  
  
CREATE INDEX idx_user_sessions_trader_id ON user_sessions(trader_id);  
CREATE INDEX idx_user_sessions_start ON user_sessions(session_start);
```

Table Relationships and Joins

Common Query Patterns

1. Get all pricing requests for a trader with results:

```
SELECT  
    pr.request_id,  
    pr.request_timestamp,  
    oc.symbol,  
    oc.strike_price,  
    oc.option_type,  
    pr.bsm_price,  
    pr.pareto_price,  
    pc.pareto_vs_bsm_pct  
FROM pricing_requests pr  
JOIN option_contracts oc ON pr.contract_id = oc.contract_id  
LEFT JOIN price_comparisons pc ON pr.request_id = pc.request_id  
WHERE pr.trader_id = $1  
ORDER BY pr.request_timestamp DESC;
```

2. Get market data with recent pricing activity:

```
SELECT  
    md.symbol,  
    md.spot_price,  
    md.data_date,  
    COUNT(pr.request_id) as daily_requests,  
    AVG(pr.bsm_price) as avg_bsm_price,  
    AVG(pr.pareto_price) as avg_pareto_price  
FROM market_data md  
LEFT JOIN pricing_requests pr ON md.market_data_id = pr.market_data_id  
WHERE md.data_date = CURRENT_DATE  
GROUP BY md.symbol, md.spot_price, md.data_date;
```

3. Trader activity summary:

```
SELECT  
    t.username,  
    t.full_name,
```

```
COUNT(pr.request_id) as total_requests,  
COUNT(CASE WHEN pr.request_timestamp >= CURRENT_DATE THEN 1 END) as  
today_requests,  
MAX(pr.request_timestamp) as last_request  
FROM traders t  
LEFT JOIN pricing_requests pr ON t.trader_id = pr.trader_id  
GROUP BY t.trader_id, t.username, t.full_name  
ORDER BY total_requests DESC;
```

Performance Considerations

1. **Indexing Strategy:** Indexes on frequently queried columns (trader_id, timestamps, symbols)
2. **Partitioning:** Consider partitioning pricing_requests table by date for better performance
3. **Connection Pooling:** Use connection pooling for the web application
4. **Query Optimization:** Regular EXPLAIN ANALYZE on common queries

Data Retention Policy

```
-- Archive old pricing requests (older than 1 year)  
CREATE TABLE pricing_requests_archive (LIKE pricing_requests INCLUDING  
ALL);  
  
-- Scheduled job to move old data  
INSERT INTO pricing_requests_archive  
SELECT * FROM pricing_requests  
WHERE request_timestamp < CURRENT_DATE - INTERVAL '1 year';  
  
DELETE FROM pricing_requests  
WHERE request_timestamp < CURRENT_DATE - INTERVAL '1 year';
```

NoSQL Alternative (For Reference)

If NoSQL were chosen (MongoDB example), the document structure would be:

Trader Document

```
{  
  "_id": ObjectId("..."),  
  "username": "trader001",  
  "email": "trader001@firm.com",  
  "profile": {  
    "fullName": "John Smith",  
    "organization": "ABC Trading",  
    "isActive": true  
  },  
  "createdAt": ISODate("2024-01-01T00:00:00Z"),  
  "lastLogin": ISODate("2024-01-15T10:30:00Z")  
}
```

Pricing Request Document

```
{
  "_id": ObjectId("..."),
  "traderId": ObjectId("..."),
  "requestTimestamp": ISODate("2024-01-15T14:30:00Z"),
  "optionDetails": {
    "symbol": "SPY",
    "strikePrice": 420.00,
    "expirationDate": ISODate("2024-03-15T00:00:00Z"),
    "optionType": "CALL"
  },
  "marketData": {
    "spotPrice": 415.50,
    "riskFreeRate": 0.0525,
    "dataDate": ISODate("2024-01-15T00:00:00Z")
  },
  "inputParameters": {
    "volatility": 0.20,
    "timeToExpiry": 0.16667,
    "tailIndexAlpha": 3.0,
    "anchorStrike": 410.00,
    "anchorPrice": 8.50
  },
  "results": {
    "bsmPrice": 7.25,
    "paretoPrice": 8.75,
    "marketReferencePrice": 8.50,
    "comparisons": {
      "paretoVsBsmPct": 20.69,
      "marketVsBsmPct": 17.24
    }
  },
  "status": "COMPLETED"
}
```

Conclusion

For this option pricing application with 100 traders and 2,000 daily requests, **PostgreSQL** is the recommended database choice due to its:

- Strong consistency and ACID properties required for financial data
- Excellent performance for the expected load
- Rich querying capabilities for analytics and reporting
- Mature ecosystem and tooling
- Cost-effectiveness for this scale

The proposed schema provides clear separation of concerns, maintains data integrity, and supports efficient querying patterns while being easily scalable for future growth.

